

INSTRUCTION SET WITH MASM

MICROPROCESSORS & INTERFACING (CS F241)

Dr. Gargi Alavani
Department of CS & IS



Interrupts

- **Hardware-generated CALL** (externally derived from a hardware signal) OR
- **Software-generated CALL** (internally derived from the execution of an instruction or by some other internal event)
- At times, an internal interrupt is called an **Exception**.
- Either type **interrupts the program** by calling an **interrupt service procedure (ISP) or interrupt handler**

Exercise

Write a program to take two numbers as input from the user and print its addition and subtraction results.

Interrupts – Int 21h

Function 1- Character input with echo

Action:

Reads a character from the standard input device and echoes it to the standard output device.

If no character is ready it waits until one is available.

I/O can be re-directed, but prevents detection of EOF.

On entry:

AH = 01h

Returns:

AL = 8 bit data input

Interrupts – Int 21h

Function 2 - Character output

Action:

Outputs a character to the standard output device. I/O can be re-directed, but prevents detection of 'disc full'.

On entry:

AH = 02h

DL = 8 bit data (usually ASCII character)

Returns:

Nothing

Interrupts – Int 21h

Function 08- Character input with no echo

Action:

Reads a character from the standard input device without copying it to the display.

If no character is ready it waits until one is available.

On entry:

AH = 08h

Returns:

AL = 8 bit data input

Interrupts – Int 21h

Function 09- Output character string

Action:

Writes a string to the display.

On entry:

AH = 09h

DS:DX = segment:offset of string

Returns:

Nothing

Notes:

The string must be terminated by the \$ character (24h), which is not transmitted. Any ASCII codes can be embedded within the string.

Interrupts – Int 21h

Function 0Ah - Buffered input

- Action:** Reads a string from the current input device up to and including an ASCII carriage return (0Dh), placing the received data in a user-defined buffer. Input can be redirected, but this prevents detection of EOF.
- On entry:** AH = 0Ah
DS:DX = segment:offset of string buffer
- Returns:** Nothing
- Notes:** The first byte of the buffer specifies the maximum number of characters it can hold (1 to 255). This value must be supplied by the user. The second byte of the buffer is set by DOS to the number of characters actually read, excluding the terminating RETURN. If the buffer fills to one less than its maximum size, the bell is sounded and subsequent input is ignored.

If a CTRL-C is detected, an INT 23h is executed. Normal DOS keyboard editing is supported during input.

When you read from keyboard using:

```
MOV AH, 01H  
INT 21H
```

If user presses 5

AL = ASCII of '5' = **35H**, not numeric 05.

Method 1 Subtract 30H (standard and most used)

MOV AH, 01H

INT 21H

SUB AL, 30H ; convert to numeric

Works for digits 0 – 9

Method 2 — Subtract '0' (same but more readable)

```
MOV AH, 01H  
INT 21H  
SUB AL, '0'
```

Assembler replaces '0' with 30H

Method 3 — Mask lower 4 bits (bitwise method)

```
MOV AH, 01H  
INT 21H  
AND AL, 0FH
```

E.g.

'7' = 37H

37H AND 0FH = 07

- Fast method, Input has to be a digit

Method 4 — For multi-digit keyboard input (string input)

Input: 123

result = result $\times$ 10 + digit

MOV SI, OFFSET buffer+2

MOV AX, 0; for each ASCII digit in buffer

MOV AL, [SI]

SUB AL, '0' ; convert digit

MOV BL, 10

MUL BL ; result $\times$ 10

ADD AX, digit

Example 1

Write a 8086 program to take two numbers (single digit) as input from the user and print its addition and subtraction results.

How to enter two digit number?

```
.MODEL SMALL
```

```
.STACK 100H
```

```
.DATA
```

```
msg1 DB 'Enter first number (0-9): $'
```

```
msg2 DB 0DH,0AH,'Enter second number (0-9): $'
```

```
msg3 DB 0DH,0AH,'Addition = $'
```

```
msg4 DB 0DH,0AH,'Subtraction = $'
```

```
num1 DB ?
```

```
num2 DB ?
```

```
res DB ?
```

First Number

```
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

; ----- INPUT FIRST NUMBER -----
    MOV AH, 09H
    LEA DX, msg1
    INT 21H

    MOV AH, 01H    ; read char
    INT 21H
    SUB AL, 30H    ; ASCII to number
    MOV num1, AL
```

Second Number

```
; ----- INPUT SECOND NUMBER -----  
MOV AH, 09H  
LEA DX, msg2  
INT 21H  
  
MOV AH, 01H  
INT 21H  
SUB AL, 30H  
MOV num2, AL
```

Addition

```
; ----- ADDITION -----  
MOV AL, num1  
ADD AL, num2  
MOV res, AL  
  
MOV AH, 09H  
LEA DX, msg3  
INT 21H  
  
MOV AL, res  
ADD AL, 30H      ; number to ASCII  
MOV DL, AL  
MOV AH, 02H  
INT 21H
```

Subtraction

; ----- SUBTRACTION -----

```
MOV AL, num1  
SUB AL, num2  
MOV res, AL
```

```
MOV AH, 09H  
LEA DX, msg4  
INT 21H
```

```
MOV AL, res  
ADD AL, 30H  
MOV DL, AL  
MOV AH, 02H  
INT 21H
```

Exiting the Code

```
; ----- EXIT -----  
    MOV AH, 4CH  
    INT 21H  
  
MAIN ENDP  
END MAIN
```

Example 2

Write an 8086 program to input two multi-digit numbers from the keyboard, convert them to numeric form, and display their sum.

Macros

- A macro is a group of instructions that perform one task, just as a procedure performs one task.
- Procedure is accessed via a CALL instruction, whereas all instructions defined in the macro is inserted in the program at point of usage
- Creating a macro is very similar to creating a new opcode
- Macro executes faster than procedure because there is no CALL or RET
- Assembler places the instruction when MACRO is invoked

Example

```

                                MOVE    MACRO  A, B
                                    PUSH    AX
                                    MOV     AX, B
                                    MOV     A, AX
                                    POP     AX
                                    ENDM

                                MOVE VAR1,VAR2    ;;move VAR2 into VAR1

0000  50                      1      PUSH    AX
0001  A1 0002 R              1      MOV     AX,VAR2
0004  A3 0000 R              1      MOV     VAR1,AX
0007  58                      1      POP     AX

                                MOVE    VAR3,VAR4    ;;move VAR4 into VAR3

0008  50                      1      PUSH    AX
0009  A1 0006 R              1      MOV     AX,VAR4
000C  A3 0004 R              1      MOV     VAR3,AX
000F  58                      1      POP     AX

```

;; is customary in a Macro

Example 1

```
PRINT MACRO message  
mov ah, 09h  
lea dx, message  
int 21h  
ENDM
```

```
.data  
msg1 db 'Hello, World!$'  
  
.code  
PRINT msg1 ; Calls the  
macro, replacing it with  
mov/lea/int instructions
```

Example 2

```
READ_NUMBER MACRO numVar
    mov ah, 01h ; Read first digit
    int 21h
    sub al, 30h ; Convert ASCII to number
    mov bl, al ; Store first digit

    mov ah, 01h ; Read second digit
    int 21h
    sub al, 30h ; Convert ASCII to number
    mov bh, al ; Store second digit

    mov al, bl ; First digit
    mov ah, 10
    mul ah ; Multiply first digit by 10
    add al, bh ; Add second digit

    mov numVar, al ; Store the result
ENDM
```



BITS Pilani
K K Birla Goa Campus

THANK YOU

